

ORBIS SCHEDULER

Use Case Specification

Multi-Tenant Service Booking & Scheduling Platform

Author	Hussain Jameel
Subject	ITA602 — Advanced IT Work Integrated Learning
Institution	Torrens University Australia
Use Cases	15, across 3 actor categories — Customer, Business Owner, Platform Administrator
Document Status	Reconciled against the shipped codebase — 8 corrections applied, flagged inline as Revision notes
Last Updated	18 August 2026

Table of Contents

1. Introduction.....	3
2. Actor Overview.....	4
3. Use Case Index.....	4
4. Customer Use Cases.....	5
UC1 — Customer Initiates Booking from Business Website.....	5
UC2 — Customer Fills Business-Specific Booking Form.....	6
UC3 — Booking Submission & Pending Review.....	7
5. Business Owner Use Cases.....	9
UC4 — Business Owner Registers and Sets Up Account.....	9
UC5 — Business Owner Resets Forgotten Password.....	10
UC6 — Business Owner Reviews and Decides on a Pending Booking.....	12
UC7 — Business Owner Sets Availability and Booking Duration.....	13
UC8 — Business Owner Builds Their Custom Booking Form.....	14
UC9 — Business Owner Views Dashboard and Manages Bookings.....	16
UC10 — Business Owner Gets Their Public Booking Link and Widget Snippet.....	17
6. Admin Use Cases.....	19
UC11 — Admin Logs In.....	19
UC12 — Admin Reviews and Approves Business Registrations.....	19
UC13 — Admin Views Platform Overview.....	21
UC14 — Admin Views All Businesses.....	21
UC15 — Admin Suspends or Activates a Business.....	22
7. Revision Log.....	24

1. Introduction

This document specifies the complete set of use cases for Orbis Scheduler, a multi-tenant SaaS-style service booking and scheduling platform. Each use case describes a single interaction between an actor and the system, including the main flow, alternate paths, exceptions, and key technical notes.

The system supports three actor categories — Customer, Business Owner, and Platform Administrator — across a total of fifteen use cases. Together, these specifications define the full functional behaviour the system implements for the MVP scope of the ITA602 project.

All use cases align with the finalised seven-table relational schema and reflect the request-and-approval booking model where customer submissions are reviewed by the business owner before confirmation. Business owner registration follows the same approval pattern: new businesses are placed in a pending state and granted access only after admin approval.

Revision note: *This revision reconciles the use case specification against the shipped codebase (docs/IMPLEMENTATION_NOTES.md and the live routes). Eight corrections were applied — three response-message/status-code fixes, one collapsed error-handling flow, one schema-level rewrite (UC5), and one architecture-level rewrite (UC10) — each marked inline where it occurs. See Section 7 for the full list.*

2. Actor Overview

Actor	Description	Authentication
Customer	End user who books an appointment via a business's website or public booking link. Does not require an account.	None — name and email captured per booking
Business Owner	Operator of a service business who manages their availability, booking form, and incoming bookings through a private dashboard. Must be approved by an administrator before gaining access.	JWT-based login
Platform Administrator	Operator of the Orbis Scheduler platform itself. Has cross-tenant visibility for support, approving new business registrations, and platform-management tasks.	JWT-based login with admin role

3. Use Case Index

UC	Title	Primary Actor
UC1	Customer Initiates Booking from Business Website	Customer
UC2	Customer Fills Business-Specific Booking Form	Customer
UC3	Booking Submission & Pending Review	Customer
UC4	Business Owner Registers and Sets Up Account	Business Owner
UC5	Business Owner Resets Forgotten Password	Business Owner
UC6	Business Owner Reviews and Decides on a Pending Booking	Business Owner
UC7	Business Owner Sets Availability and Booking Duration	Business Owner
UC8	Business Owner Builds Their Custom Booking Form	Business Owner
UC9	Business Owner Views Dashboard and Manages Bookings	Business Owner
UC10	Business Owner Gets Their Public Booking Link and Widget Snippet	Business Owner
UC11	Admin Logs In	Administrator
UC12	Admin Reviews and Approves Business Registrations	Administrator
UC13	Admin Views Platform Overview	Administrator
UC14	Admin Views All Businesses	Administrator
UC15	Admin Suspends or Activates a Business	Administrator

4. Customer Use Cases

This section covers the three customer-facing use cases — the full journey from arriving on a booking page through to submitting a booking request that awaits business confirmation.

Use Case 1: Customer Initiates Booking from Business Website

Actor	Customer (no account, no login)
Trigger	Customer clicks the "Book Appointment" button on Business ABC123's own website
Preconditions	Business ABC123 is registered on the platform and has been approved by an administrator. Business ABC123 has embedded the widget (or shareable link) on their website. Business ABC123 has configured availability and an active booking form.

Revision note: Alternate flows A2–A4 below were corrected. The original spec described three different messages for a pending, rejected, or suspended business; the shipped platform returns the same 404 used for a nonexistent business (A1) in all four cases — see [IMPLEMENTATION_NOTES.md §3](#).

Main Flow

- Customer is browsing Business ABC123's website.
- Customer clicks the "Book Appointment" button.
- Browser opens the booking page at `orbisscheduler.app/book/[ABC123-uuid]`.
- Platform receives the request and extracts the business UUID from the URL.
- Backend queries the database: `SELECT * FROM businesses WHERE id = [uuid] AND approval_status = 'approved' AND is_active = true`.
- Backend queries the business's active booking form and form fields: `SELECT * FROM booking_forms WHERE business_id = [uuid] AND is_active = true`, then `SELECT * FROM form_fields WHERE form_id = [form_id] ORDER BY display_order`.
- Platform renders the public booking page showing the business name, description, and the rendered booking form alongside an availability calendar.
- Customer views the page, ready to fill in the form and select a slot.

Postcondition: Customer sees Business ABC123's booking form and availability only — no other tenant's data is visible.

Alternate Flows

A1 — Business UUID doesn't exist (invalid/tampered URL)

- Step 5: Backend finds no matching business.
- Platform returns 404 and shows a "Business not found." error page.
- No data is leaked.

A2 — Business exists but registration is still pending approval

- Step 5: Business record exists but `approval_status = 'pending'`.
- Platform returns the same 404 "Business not found." response as A1 — the customer is never told a pending business exists.
- No booking action possible.

A3 — Business registration was rejected

- Step 5: Business record exists but `approval_status = 'rejected'`.
- Platform returns the same 404 "Business not found." response as A1.
- The rejection reason is never exposed to the customer.

A4 — Business account is deactivated/suspended by admin

- Step 5: Business record exists, is approved, but `is_active` = false.
- Platform returns the same 404 "Business not found." response as A1.

A5 — Business has not yet configured availability

- Step 7: No availability rules exist yet for this business.
- Platform renders the booking form but shows "No slots available yet" in the calendar area.

Exceptions

E1 — Database/server error

- Platform shows generic error page and logs the incident.
- No partial or broken data shown to the customer.

Technical Notes

- A1 through A4 are deliberately indistinguishable to the customer — a nonexistent, pending, rejected, or suspended business all return the identical 404 "Business not found." response. Returning different messages for each case would let an attacker enumerate which business UUIDs exist and infer a business's internal approval/activation status.

Use Case 2: Customer Fills Business-Specific Booking Form

Actor	Customer (no account, no login)
Trigger	Customer has landed on Business ABC123's booking page (from UC1) and proceeds to fill in their booking details
Preconditions	Business ABC123 has an active booking form configured in the <code>booking_forms</code> table. Form fields are defined in the <code>form_fields</code> table with type, label, required flag, and options where applicable. All fields are shown together — there is no conditional field logic.

Revision note: A2 and A3 below were corrected to match the actual validation message and the actual slot-race wording used across UC2/UC3 — see `IMPLEMENTATION_NOTES.md §3`.

Main Flow

- Platform renders every form field in one view based on `form_fields.display_order` — text fields, textareas, dropdowns, checkboxes and radio buttons. Field options (for dropdown, radio, checkbox) are read from the JSONB `options` column.
- Customer fills in the fields in any order. No field changes appearance based on what is selected in another field.
- Customer selects a date from the calendar. Platform calculates available slots on demand from `availability_rules` for that day of week, minus any bookings already held (pending or approved) for that date.
- Customer picks an available slot.
- Customer enters their name, email, and optionally phone number (these are universal booking fields stored on the `bookings` table itself).
- Customer reviews their submission and clicks "Submit booking request".
- Platform validates: all required form fields are filled, dropdown/radio/checkbox values match the predefined options, the email is in a valid format, and the selected slot falls within the business's `booking_window_days` configuration.
- On successful validation, control passes to UC3 for actual booking submission.

Postcondition: All form input is validated and a booking submission is ready to be created in the database, with field responses keyed against `form_field` IDs.

Alternate Flows

A1 — Customer leaves a required field empty

- Step 7: Validation fails.
- Platform highlights missing fields, customer cannot proceed until resolved.

A2 — Customer submits a value not in the field's options list

- Step 7: Validation fails for that field.
- Platform rejects the submission with "\${field.label} must be one of the provided options".

A3 — Customer's selected slot becomes unavailable while filling the form

- Step 4: Another customer holds the same slot via a pending or approved booking.
- Platform detects the conflict at submission time and returns the customer to slot selection.
- Customer is shown "This slot was just taken. Please select another time." — the same message UC3's real race path uses, so the two cases are indistinguishable to the customer.

A4 — Business hasn't customised the form yet

- Default form fields (Name, Email, Phone) are still rendered from the default form_fields rows created at registration.
- Booking proceeds normally with just those fields.

Exceptions

E1 — Form schema is corrupted or invalid

- Platform fails to render the form safely.
- Generic error shown, incident logged, customer asked to try again later.

Use Case 3: Booking Submission & Pending Review

Actor	Customer (no account, no login)
Trigger	Customer has submitted the booking form and selected a slot (from UC2)
Preconditions	Selected slot was still available at the moment of submission. Form data passed validation.

Revision note: A1's quoted message below was corrected to match the single string the shipped code actually returns for a slot-taken race — the original spec quoted different wording here than in UC2 A3 for what is, on the backend, the exact same event.

Main Flow

- Customer submits the completed form and selected slot.
- Platform performs a final availability check to re-verify the slot hasn't been taken between selection and submission.
- Platform creates a row in the bookings table with status = 'pending', business_id (denormalised for direct tenant queries), form_id, booking_date, booking_time, customer_name, customer_email and customer_phone.
- Platform creates one row in booking_field_values for each filled form field, storing booking_id, form_field_id and value (as text regardless of original field type).
- Platform locks the slot immediately — no other customer can select it while the booking sits at 'pending' status.
- Platform displays the Booking Request Sent screen to the customer, showing: "Booking request sent. [Business Name] will confirm shortly", the date and time requested, the booking reference ID, and a read-only summary of the form details they submitted.
- Platform sends a "Request Received" email to the customer's submitted email address (via Nodemailer).

- Platform sends an "Action Required" email/notification to the business owner with the booking details and a prompt to review it in their dashboard.
- Booking appears in the business owner's dashboard under the Pending Requests tab.

Postcondition: Booking exists with status 'pending', the slot is locked, and the decision now sits with the business owner (handled in UC6).

Alternate Flows

A1 — Slot was taken between selection and submission (race condition)

- Step 2: Re-check finds the slot is no longer available because another pending or approved booking already holds it. A Postgres partial unique index on bookings (businessId, bookingDate, bookingTime) — filtered to pending/approved rows — is what actually closes this race at the database level, not just the application-level re-check.
- Platform does not create the booking.
- Customer is shown "This slot was just taken. Please select another time." — the same message UC2 A3's pre-submission check uses, deliberately, so a genuine race and an already-taken slot are indistinguishable to the customer.
- Customer is returned to slot selection — form data they already entered is preserved.

A2 — Email delivery fails (customer or business)

- Step 7/8: Nodemailer returns an error.
- Booking is still created as 'pending' — email failure does not block the request.
- Platform logs the failed email for retry.
- Customer still sees the on-screen "Request Sent" confirmation regardless of email status.

Exceptions

E1 — Database failure during booking creation

- Step 3 or 4 fails.
- Booking creation is wrapped in a transaction — if the booking_field_values inserts fail, the bookings row is rolled back.
- Customer is shown a generic error and asked to try again.

Technical Notes

- Locking the slot at 'pending' prevents two customers requesting the same time while one is awaiting review. A sensible expiry rule (e.g. auto-reject after 24–48 hours unactioned) can be added as a future improvement.
- Slot availability is calculated on demand from availability_rules at request time. Pre-storing slots is intentionally avoided to keep the database small and always accurate.
- All field values are stored as text in booking_field_values.value regardless of the original field_type. Type-aware rendering is handled at display time based on the linked form_field.

5. Business Owner Use Cases

This section covers the seven business-owner use cases, beginning with registration and the admin-approval gate, account recovery via password reset, configuring availability and forms, and the ongoing management of bookings.

Use Case 4: Business Owner Registers and Sets Up Account

Actor	Business Owner (new user)
Trigger	Business owner visits the platform's landing page and clicks "Sign Up"
Preconditions	Business owner has a valid email address. Business owner does not already have an account with this email.

Main Flow

- Business owner navigates to /register.
- Platform displays a registration form with the following fields: business name, owner's full name, owner email (for login), password (with confirmation), business contact phone, business description (optional), website URL (optional).
- Business owner fills in all required fields and clicks "Create Account".
- Platform validates the input: all required fields are filled, email is in a valid format, password meets minimum strength requirements (8+ characters), and email is not already registered in the users table.
- Platform creates a row in the users table with role = 'owner' and a bcrypt-hashed password.
- Platform creates a row in the businesses table linked to that user, with auto-generated UUID, a unique URL-friendly slug derived from the business name, and approval_status = 'pending'.
- Platform creates default booking_forms and form_fields rows (Name, Email, Phone) for the new business so the form is functional immediately upon approval.
- Platform displays a "Registration submitted. Awaiting admin approval." screen confirming the submission. The owner is not logged in at this point — no JWT is issued at registration.
- Platform sends a notification email to the administrator that a new business registration is awaiting review.
- Platform sends a confirmation email to the business owner acknowledging the submission and explaining the next steps.

Postcondition: A users row and a businesses row both exist. The business sits at approval_status = 'pending'. The owner cannot log in or access any dashboard until an admin approves the registration (handled in UC12).

Alternate Flows

A1 — Email already registered

- Step 4: Email is found in the users table.
- Platform displays "An account with this email already exists. Please log in instead."
- Provides a link to the login page.

A2 — Validation fails (weak password, invalid email, missing fields, slug conflict)

- Step 4: Validation errors found.
- Platform highlights the specific fields with inline error messages.
- If the business name's slug conflicts with an existing business, platform appends a numeric suffix automatically.
- Owner corrects and resubmits.

A3 — Business owner tries to log in while registration is still pending

- Owner navigates to /login and enters credentials.
- Platform validates credentials successfully but checks businesses.approval_status.
- If status is 'pending', login is blocked with the message "Your registration is still under review. We will email you once approved."

- No JWT is issued; no dashboard access is granted.

A4 — Business registration is rejected by admin

- Admin sets approval_status = 'rejected' with a rejection reason (UC12).
- On next login attempt, owner is shown "Your registration was not approved." along with the rejection reason.
- Owner cannot reapply with the same email unless an admin clears the rejection.

A5 — Business registration is approved by admin

- Admin sets approval_status = 'approved' (UC12).
- Platform sends an approval email to the owner with a link to log in.
- On next login, platform issues a JWT containing the business UUID and role.
- Owner is redirected to the dashboard.
- Owner can now navigate the dashboard freely.

Exceptions

E1 — Database error during account creation

- Step 5, 6 or 7 fails.
- The whole registration is wrapped in a transaction — no partial state is left behind.
- Owner sees a generic error and is asked to try again.

E2 — Notification email to admin or owner fails

- Steps 9 or 10 fail.
- Registration is still created and marked pending — email failure does not block the workflow.
- Email failures are logged for retry. The admin can still discover the pending business via the admin dashboard (UC12).

Technical Notes

- Passwords are hashed with bcrypt (cost 12) — never stored or logged in plain text.
- Business UUIDs are version 4 (random) — never sequential — to prevent IDOR enumeration on the public booking URL.
- The slug field is generated from the business name (lowercased, hyphenated, suffixed if needed) and is also unique. The UUID is the canonical identifier; the slug is for human-friendly URLs.
- Default booking_forms and form_fields are pre-created at registration so that customers can still book using a minimal form even if the owner hasn't customised it after approval.
- Email verification (proving the owner controls the email address) is not in MVP scope but is flagged as a future improvement layered on top of admin approval.

Use Case 5: Business Owner Resets Forgotten Password

Actor	Business Owner (not logged in)
Trigger	Business owner clicks "Forgot password?" on the login screen
Preconditions	Owner has an account in the users table with role = 'owner'. Owner has access to the email address registered on their account.

Revision note: This use case was rewritten. The original spec described a dedicated password_reset_tokens table with hashed tokens, a used flag, 1-hour expiry, and logic to invalidate other outstanding tokens. The shipped schema has no such table — reset_token and reset_token_expires_at are two plain columns on users, the expiry is 30 minutes, and "only one token at a time" falls out naturally from there being a single column rather than from any explicit invalidation logic. See db_schema.sql and IMPLEMENTATION_NOTES.md.

Main Flow

- Business owner navigates to /login and clicks the "Forgot password?" link.
- Platform displays a "Reset password" form asking for the owner's registered email address.
- Owner enters their email and clicks "Send reset link".
- Platform checks the users table for that email.
- Platform generates a cryptographically random reset token and stores it directly on the matching users row — reset_token holds the token value and reset_token_expires_at is set 30 minutes ahead. There is no separate password_reset_tokens table; a new request simply overwrites the previous token, so only one reset link can ever be valid for a user at a time.
- Platform sends an email (via Nodemailer) containing a reset link of the form orbisscheduler.app/reset-password?token=[token]. The frontend reads the token from the URL query string automatically — the owner never types it manually.
- Platform displays a generic confirmation screen: "If this email is registered you will receive a reset link shortly." — the same message regardless of whether the email matched a real account (prevents email enumeration).
- Owner opens the email and clicks the reset link within 30 minutes.
- Platform looks up the token directly against users.reset_token and checks that reset_token_expires_at has not passed.
- Platform displays the "Set new password" screen with a new-password field.
- Owner enters a new password (8+ characters minimum) and submits.
- Platform validates the new password and, on success, updates password_hash with a fresh bcrypt hash and clears reset_token and reset_token_expires_at back to null — which is what makes the link single-use, since a cleared token can never match a later request.
- Platform displays a success screen: "Password reset successful. You can now log in." with a link to /login.
- Owner clicks through to login and signs in normally with the new password.

Postcondition: The owner's password_hash is updated, reset_token and reset_token_expires_at are cleared, and the owner can authenticate with their new password.

Alternate Flows

A1 — Email does not match any account

- Step 4: No user exists with the provided email.
- Platform still displays the same generic confirmation screen as the success case; no token is generated.
- This deliberate ambiguity prevents an attacker from probing the platform to discover which emails are registered.

A2 — Reset link has expired

- Step 10: reset_token matches but reset_token_expires_at is in the past.
- Platform shows "Invalid or expired reset link."
- Owner returns to the "Forgot password" page and starts over.

A3 — Reset link is invalid, already used, or tampered

- Step 10: no user row has a matching reset_token. This single check covers all three cases — a link that's already been used (a successful reset clears reset_token to null), a stale link superseded by a newer request (overwritten in step 5), and a tampered token (never matches any stored value).
- Platform shows the same generic "Invalid or expired reset link." error in every case — no information is leaked about whether the token ever existed.

A4 — New password fails validation

- Step 12: password is under 8 characters.
- Platform highlights the issue inline; the reset is not completed.
- The token remains valid until it expires or a valid reset is completed.

Exceptions

E1 — Email delivery fails when sending the reset link

- Step 6: Nodemailer returns an error.
- The token is still saved on the user's row — the owner can request again, which overwrites it with a fresh token.
- Failure is logged for diagnostic purposes. The generic confirmation screen is still shown regardless of email outcome.

E2 — Database error during password update

- Step 12: the password_hash update fails.
- No partial update — the password is not changed and the token is not cleared.
- Owner sees a generic error and can retry with the same link if it's still within its expiry window.

Technical Notes

- The reset token lives in two plain columns directly on users — reset_token and reset_token_expires_at — not a separate password_reset_tokens table. This is a simpler model than a hardened multi-token design with a used flag, but it means only one reset link can ever be valid per user at a time; requesting a new link silently invalidates any earlier one by overwriting it.
- Tokens expire after 30 minutes.
- The platform always returns the same generic confirmation message, and the same generic error message on failure, regardless of whether the email matched an account or the token was valid, expired, already used, or never existed — a deliberate choice to prevent both account and token enumeration.
- Resetting a password does not affect any existing bookings, availability rules, or form configurations — only password_hash, reset_token, and reset_token_expires_at on the users row are touched.
- Active JWT sessions for the owner are not automatically invalidated on password reset in MVP. Revoking active sessions on reset, and hashing the stored token, are both flagged as future hardening improvements.

Use Case 6: Business Owner Reviews and Decides on a Pending Booking

Actor	Business Owner (logged in via JWT, business is approved and active)
Trigger	Business owner opens the "Pending Requests" tab in their dashboard
Preconditions	Business is approved (approval_status = 'approved') and active (is_active = true). At least one booking exists with status = 'pending' for this business. Business owner is authenticated.

Revision note: A1 below was corrected from 403 to 404. Git history shows 403 was only ever used on a draft route deleted before ship; the live convention (and the stronger anti-enumeration choice) is 404 everywhere cross-tenant access is attempted — see IMPLEMENTATION_NOTES.md §2 and §3.

Main Flow

- Business owner navigates to Dashboard → Pending Requests.
- Platform fetches all bookings where business_id = [token's business_id] AND status = 'pending'.
- Business owner views the list — each entry shows customer name, booking date and time, and a summary of submitted form responses.
- Business owner opens a specific request to see full details, including every field response joined from booking_field_values.
- Business owner clicks Approve or Reject. Optionally writes a note into owner_notes.
- If Approved: platform updates booking status to 'approved'; the slot remains permanently locked; a "Booking approved" email is sent to the customer; the booking moves to the Upcoming Bookings tab.
- If Rejected: platform updates booking status to 'rejected'; the slot is released and becomes available again; a "Booking rejected" email is sent to the customer (including any owner_notes if provided); the booking moves to a Rejected/Past Requests tab.

Postcondition: Booking has a final status ('approved' or 'rejected'), customer has been notified, and slot availability reflects the outcome.

Alternate Flows

A1 — Business owner tries to act on a request belonging to a different business (tampered request)

- Step 5: Backend re-validates that the booking's business_id matches the JWT token's business_id.
- If mismatch, request is rejected with 404 Not Found — no status change occurs. This is deliberate: a 403 would confirm the booking exists on another tenant, while 404 gives an attacker nothing to distinguish a wrong ID from someone else's real booking.

A2 — Customer's email is invalid or bounces

- Step 6: Email fails to send.
- Status change still completes — customer not being notified doesn't block the business's decision.
- Failure is logged for retry.

Exceptions

E1 — Race condition on slot release

- When a booking is rejected, the slot is immediately released. Any other pending booking that conflicts with the same slot may now become resolvable.
- The platform does not auto-notify other pending customers of newly opened slots in MVP (future improvement).

Use Case 7: Business Owner Sets Availability and Booking Duration

Actor	Business Owner (logged in via JWT, business is approved and active)
Trigger	Business owner navigates to Dashboard → Availability (either from onboarding or main nav)
Preconditions	Business owner is authenticated. Business is approved and active.

Main Flow

- Business owner clicks "Availability" in the dashboard navigation.
- Platform fetches the business's existing availability rules: `SELECT * FROM availability_rules WHERE business_id = [token's business_id] ORDER BY day_of_week`.
- Platform renders the Availability page showing all 7 days (0 = Monday, 6 = Sunday). For each day the owner sees an Open/Closed toggle (is_available), start_time and end_time pickers, optional break_start and break_end pickers, and a slot_duration_minutes dropdown (e.g. 15, 30, 45, 60, 90, 120).
- Business owner configures their week — toggling days, setting times, picking durations.
- Business owner clicks "Save".
- Platform validates: for each open day, end_time must be after start_time; at least one day must be open; if break_start/break_end are set they must fall within start_time/end_time; slot_duration_minutes must fit at least one whole slot within the day.
- Platform replaces the full weekly schedule — one availability_rules row per day_of_week — in a single PUT request; the owner always resubmits all 7 days at once.
- Platform displays a success message: "Availability updated successfully."

Postcondition: Business's availability_rules rows are updated. The customer-facing booking page will calculate slots dynamically from this configuration on every request.

Alternate Flows

A1 — Business owner saves with all days closed

- Step 6: Validation fails.

- Platform shows: "You must have at least one open day for customers to book."
- Save is blocked.

A2 — Business owner enters end_time before start_time

- Step 6: Validation fails for that specific day.
- Platform highlights the day: "End time must be after start time."
- Save is blocked.

A3 — slot_duration_minutes exceeds the open window on an open day

- Step 6: Validation fails.
- Platform shows a message explaining the chosen duration doesn't fit the day's open window.
- Save is blocked.

A4 — Existing bookings already fall outside the new availability

- Owner has approved or pending bookings, then changes hours to exclude those slots.
- Platform allows the change but does not auto-cancel existing bookings.
- Existing bookings remain valid; only future slot generation is affected.
- Known gap: the save-time warning ("You have N existing bookings outside your new hours") described in earlier planning is not implemented — PUT /owner/availability doesn't return a count of affected bookings, so the frontend has nothing to show yet. Tracked in FUTURE_IMPROVEMENTS.md.

A5 — Business owner returns later to update availability

- Owner re-opens the Availability page.
- Platform pre-fills the form with existing availability_rules rows. A brand-new business with no saved rows defaults every day to closed, not a pre-filled starter week.
- Owner modifies and saves as in the main flow.

Exceptions

E1 — Database error during save

- Step 7 fails.
- Save is wrapped in a transaction — partial updates are rolled back.
- Owner asked to retry.

Technical Notes

- Slots are never stored. They are calculated on demand from availability_rules + existing bookings each time a customer queries availability for a date. This guarantees freshness without state-sync issues.
- day_of_week uses 0 = Monday through 6 = Sunday. This is an arbitrary but documented convention.
- Optional break_start and break_end allow a midday gap (e.g. lunch). The slot generator excludes any slot whose time falls within break_start <= time < break_end.
- Existing bookings are never silently invalidated by an availability change. The business made a commitment to those customers and the platform respects it.

Use Case 8: Business Owner Builds Their Custom Booking Form

Actor	Business Owner (logged in via JWT, business is approved and active)
Trigger	Business owner navigates to Dashboard → Booking Form (either from onboarding or main nav)
Preconditions	Business owner is authenticated. Business is approved and active. A default form (Name, Email, Phone) exists in booking_forms + form_fields from registration.

Main Flow

- Business owner clicks "Booking Form" in the dashboard navigation.
- Platform fetches the active booking form and its fields: `SELECT * FROM booking_forms WHERE business_id = [token's business_id] AND is_active = true`, then `SELECT * FROM form_fields WHERE form_id = [form_id] ORDER BY display_order`.
- Platform renders the Form Builder page with two main areas — Left side (Field List): current `form_fields` rows displayed in `display_order`, each showing label, `field_type`, `is_required`, with edit, reorder and delete controls. Right side (Add Field Panel): buttons to add a new field of each supported type — text, textarea, dropdown, checkbox, radio.
- Business owner clicks "Add Field" → picks a type → a field configuration modal opens.
- Field configuration modal asks for: label, `is_required` toggle, and for dropdown/checkbox/radio types only — a list of option strings stored in the `JSONB options` column.
- Business owner repeats this for as many fields as they need.
- Business owner can reorder fields by dragging (which updates `display_order` on each row), edit a field's properties, or delete a field. Default Name and Email fields are protected — flagged with a real `isProtected` boolean column, not string-matching on the label — from deletion or editing.
- Business owner clicks "Save Form".
- Platform validates: every field has a label; every dropdown/checkbox/radio field has at least one option in options.
- Platform persists changes — inserting new `form_fields` rows, updating modified ones, deleting removed ones, and re-sequencing `display_order` — all within a transaction.
- Platform displays a success message: "Fields reordered successfully." / "Field updated successfully." / "Field deleted successfully." depending on the action taken.

Postcondition: The booking form is updated. Future customers visiting the booking page see the new field set; historical bookings retain their original responses via `booking_field_values` referencing the original `form_field` IDs.

Alternate Flows

A1 — Business owner tries to delete or edit a protected default field (Name/Email)

- Step 7: Delete or edit is blocked with a 403.
- Platform shows a message explaining the field is required to contact the customer and cannot be removed or renamed.

A2 — Business owner tries to save with an incomplete dropdown (no options)

- Step 9: Validation fails.
- Platform highlights the problem field: "Add at least one option for this dropdown."
- Save is blocked.

A3 — Business owner tries to change a field's type after creation

- `fieldType` is immutable once a field exists — including it in an edit request returns 400.
- The Form Builder UI disables the Type selector once a field is being edited rather than created fresh, so this case is prevented at the UI level as well as the API level.

A4 — Business owner edits or deletes a field that has existing booking_field_values

- Existing bookings already reference `form_field` IDs.
- Old bookings retain their original responses; the renamed/deleted field no longer appears on the form for new bookings.

A5 — Business owner returns later to modify the form

- Owner re-opens the Form Builder.
- Platform pre-fills with the existing fields.
- Owner modifies and saves as in the main flow.

Exceptions

E1 — Database error during save

- Step 10 fails.
- The save transaction is rolled back; no partial form schema is left behind.
- Owner asked to retry.

Technical Notes

- Splitting the form into booking_forms (one row per business) and form_fields (one row per field) makes the schema queryable from SQL without needing JSON parsing for the field list. Options for choice-type fields are stored as JSONB on form_fields.options because their shape is naturally a small array of strings.
- Each form_field has a stable integer ID. booking_field_values references form_field_id, so renaming a field's label doesn't break historical responses.
- No conditional logic is supported — fields do not depend on each other (deliberate scope decision). This keeps validation simple and the customer-facing form fully static.
- Drag-and-drop reordering is implemented with dnd-kit; reordering rewrites display_order on the affected rows in a single transaction.

Use Case 9: Business Owner Views Dashboard and Manages Bookings

Actor	Business Owner (logged in via JWT, business is approved and active)
Trigger	Business owner logs in or navigates to the Dashboard home
Preconditions	Business owner is authenticated. Business is approved and active.

Revision note: A3 below was corrected from 403 to 404 — the same anti-enumeration convention corrected in UC6 A1. This is the exact passage IMPLEMENTATION_NOTES.md flags by name.

Main Flow

- Business owner logs in or clicks "Dashboard" in the main navigation.
- Platform fetches dashboard data scoped to the business: WHERE business_id = [token's business_id].
- Platform renders the Dashboard home with a greeting, stat cards, a list of recent bookings, and a setup-status nudge for owners still finishing onboarding.
- Business owner navigates to the Bookings screen to see the full list, filterable by status (pending/approved/rejected/cancelled) with real counts shown on every filter pill regardless of which one is active, searchable by customer name or email, and paginated at 25 per page.
- Business owner can filter, search, sort, and click a booking to view full details.
- From the booking detail view, the business owner can approve/reject a pending request (triggers UC6), cancel an approved booking (sets status to 'cancelled' and releases the slot), or view all submitted field responses.

Postcondition: Business owner has a complete view of their booking state and can take action on any booking.

Alternate Flows

A1 — Business has no bookings yet (brand new account)

- Step 4: All booking lists are empty.
- Platform shows an empty-state with onboarding guidance.
- Quick stats show zeros.

A2 — Business owner cancels an approved booking

- Owner clicks "Cancel booking".
- Platform asks for confirmation (and optionally a reason captured in owner_notes).
- Booking status updated to 'cancelled'.

- Slot is released and becomes available again.
- Customer receives a cancellation email.
- Booking moves from "Upcoming" tab to "Past"/"Cancelled" tab.

A3 — Business owner tries to access another business's bookings (URL tampering)

- Backend always filters by JWT's business_id — URL parameters are ignored on protected routes.
- Any attempt to access /api/owner/bookings/[some-other-id] returns 404 Not Found, not 403 — a 403 would confirm the booking exists on another tenant; 404 gives an attacker nothing to distinguish a wrong ID from someone else's real booking.

Exceptions

E1 — Database error fetching bookings

- Platform shows error state with retry option.
- Stats default to "—" rather than zero (to distinguish "no data" from "data unavailable").

Technical Notes

- Pagination is designed in from day one — bookings page 25 at a time rather than loading everything.
- Real-time updates are not in MVP scope — a manual "Refresh" button is sufficient. WebSockets or polling can be added later.
- Tenant isolation is enforced at every query: every SELECT on bookings includes WHERE business_id = [token's business_id]. The business_id column is denormalised onto bookings for direct tenant queries without joining through booking_forms.
- The status-filter counts are computed via a separate, status-blind groupBy query, so filtering to one status still shows real numbers on the other filter pills instead of zeroing them out.

Use Case 10: Business Owner Gets Their Public Booking Link and Widget Snippet

Actor	Business Owner (logged in via JWT, business is approved and active)
Trigger	Business owner navigates to Dashboard → Share & Embed (either from onboarding or main nav)
Preconditions	Business owner is authenticated. Business is approved and active. Business has a UUID assigned from registration.

Revision note: Section B and the Technical Notes below were rewritten. The original spec described an iframe HTML snippet; the shipped embed mechanism is a <script> tag opening a Shadow-DOM-isolated modal widget (widget.js) — not an iframe at all. A1 was also updated to reflect where the "no availability" check actually happens. See IMPLEMENTATION_NOTES.md §3-§4.

Main Flow

- Business owner clicks "Share & Embed" in the dashboard navigation.
- Platform fetches the business's UUID from the token and constructs the public-facing URL: orbisscheduler.app/book/[uuid]. The public origin is derived from the incoming request's headers rather than a hardcoded env var.
- Platform renders the Share & Embed page with three sections — Section A: Your Booking URL (full URL in a read-only field, Copy button, Open in new tab button); Section B: Embed on your website (a <script src=".../widget.js" data-business-id="..."></script> snippet with Copy button — the script auto-injects a "Book Now" button that opens the booking form in a Shadow-DOM-isolated modal overlay directly on the host page, or can be triggered manually via window.Orbis.open()); Section C: QR code (QR code image with Download button).

- Business owner copies whatever they need — the URL to share via WhatsApp/email/social, the script snippet for their website, or downloads the QR code for print materials.
- Business owner can click "Test booking page" to open the live customer-facing booking page in a new tab.

Postcondition: Business owner has everything they need to start receiving bookings; the platform is fully configured for use.

Alternate Flows

A1 — Business owner clicks "Test booking page" but hasn't configured availability yet

- Before opening the live page, the Share & Embed screen itself makes an on-click GET /owner/availability call and shows a dismissible banner warning that availability isn't configured yet.
- Owner is prompted to complete availability setup first.

A2 — Business owner hasn't customised the booking form yet

- Live page still works — the default form (Name, Email, Phone) is used from the default booking_forms + form_fields rows.
- No blocking error; the platform still functions with defaults.

A3 — Business owner regenerates their UUID (future feature, not MVP)

- Not supported in MVP. The UUID is permanent for the business's lifetime.
- Flagged as a future improvement for cases like security concerns or rebranding.

Exceptions

E1 — QR code generation fails

- Page still renders normally.
- QR section shows a message explaining the QR code is unavailable and to use the URL or script snippet instead.

Technical Notes

- Embed mechanism: a plain IIFE vanilla-JS widget (widget.js), isolated in a Shadow DOM so the host site's CSS can't leak in or clash with it. It supports two modes: auto-injecting a booking button, or being opened programmatically via window.Orbis.open(). This was chosen over an iframe because a plain iframe gets visually squeezed by whatever layout surrounds it on the host page — the modal overlay approach isn't constrained by the embedding site's layout.
- QR code generation uses qrcode.react on the frontend. No external service needed.
- The embed code block is styled as a fixed dark code-editor surface regardless of the app's own light/dark theme — the one deliberate exception to always using theme tokens.
- No customer authentication is needed on the public URL — safe because (a) UUIDs prevent enumeration, (b) the public booking page only exposes the form structure and available slots (no sensitive data), and (c) submissions are write-only from the customer's perspective.

6. Admin Use Cases

This section covers the five administrator use cases. The admin role provides cross-tenant visibility for platform operation, vetting of new business registrations, and ongoing tenant management — distinct from the tenant-scoped access of business owners.

Use Case 11: Admin Logs In

Actor	Platform Administrator
Trigger	Admin navigates to the login page
Preconditions	Admin account exists in the users table with role = 'admin'. Admin has valid credentials.

Main Flow

- Admin navigates to /login — the same login route and screen owners use.
- Admin enters credentials and submits.
- Platform validates: email exists in users table, password matches (bcrypt-compared).
- Platform issues a JWT containing user_id and role. The role in the token — not a separate admin login route — determines what the owner sees next.
- Admin is redirected to the admin dashboard.

Postcondition: Admin is authenticated and can access admin-only routes.

Alternate Flows

A1 — *Wrong password or unknown email*

- Step 3: Credentials don't match.
- Login fails with the generic "Invalid credentials." message — no indication of whether the email or the password was wrong.

A2 — *Admin already logged in*

- Existing valid JWT detected on visiting /login.
- Auto-redirect to the admin dashboard.

Exceptions

E1 — *Database error during login*

- Platform shows a generic error and asks the admin to retry.

Technical Notes

- For MVP, the admin account is created manually via prisma/seed.ts. There is no admin registration route — admins are never self-registered.
- Admin and owner share the same login route; only the role encoded in the returned JWT differs.

Use Case 12: Admin Reviews and Approves Business Registrations

Actor	Platform Administrator (logged in via JWT)
Trigger	Admin opens the "Pending Registrations" view in the admin dashboard, or arrives via the notification email
Preconditions	Admin is authenticated with role = 'admin'.

At least one business exists with approval_status = 'pending'.
--

Main Flow

- Admin navigates to Admin Dashboard → Businesses, filtered to pending.
- Platform fetches all businesses where approval_status = 'pending', joined with their owner from the users table.
- Admin views the list — each entry shows business name, owner name and email, contact phone, business description, website URL, and the time the registration was submitted.
- Admin clicks a business to see full registration details.
- Admin makes a decision: Approve or Reject. For a rejection, the admin enters a rejection reason that will be shown to the owner.
- If Approved: platform sets approval_status = 'approved' on the businesses row. An approval email is sent to the owner with a link to log in.
- If Rejected: platform sets approval_status = 'rejected' and stores the rejection reason. A rejection email is sent to the owner with the reason. The owner cannot reapply with the same email unless an admin clears the rejection.

Postcondition: Business has been moved from 'pending' to either 'approved' (full access granted) or 'rejected' (access blocked, with reason).

Alternate Flows

A1 — Admin opens a registration that has already been actioned

- Step 4: Business's approval_status is no longer 'pending'.
- Platform shows a banner explaining the registration has already been approved or rejected.
- Approve/Reject buttons are disabled.

A2 — Admin needs more information before deciding

- Admin can leave the registration as 'pending' and return later.
- Admin can contact the owner via their registered email outside the platform (no in-platform messaging in MVP).

A3 — Admin tries to reject without providing a reason

- Step 5: Submission is blocked.
- Platform shows a message asking the admin to provide a reason so the business owner understands why.

A4 — Email notification to the owner fails

- Step 6/7: Email send fails.
- Status change still completes — the approval/rejection is persisted regardless.
- Email failure is logged for retry. The owner will discover the status next time they attempt to log in.

Exceptions

E1 — Database error during status update

- Step 6/7 fails.
- No partial update — the approval_status change and rejection reason write happen in a single transaction.
- Admin asked to retry.

Technical Notes

- This is the only point where a business's approval_status transitions from 'pending' to 'approved' or 'rejected'. There is no "unapprove" workflow — once approved, a business is governed by the is_active flag (UC15) rather than approval_status.
- Approval is irreversible in MVP. If an approved business needs to be removed, the admin suspends it (UC15) rather than reverting approval.
- No audit log is kept of who approved/rejected which business and when. Flagged as a future improvement.
- Admin approval is the only safeguard against fake or low-quality businesses appearing on the platform in MVP. Email verification, CAPTCHA, and business identity verification are future improvements layered on top.

Use Case 13: Admin Views Platform Overview

Actor	Platform Administrator (logged in via JWT)
Trigger	Admin lands on the admin dashboard after login
Preconditions	Admin is authenticated with role = 'admin'.

Main Flow

- Admin opens the dashboard.
- Platform fetches aggregate stats across all businesses (no tenant filter): total businesses registered, breakdown by approval_status (pending/approved/rejected), total active/suspended businesses, total bookings lifetime, and bookings in the last rolling 7 days.
- Platform renders the Admin Overview with an identity block (fetched via GET /api/admin/me — the logged-in admin's own name and email), stat cards, and a highlighted count of pending business registrations linking to UC12.

Postcondition: Admin has a top-level view of platform health and activity, with pending registrations surfaced as a priority.

Alternate Flows

A1 — Platform has zero businesses (fresh deployment)

- All stats show 0.
- Empty state shown.

A2 — There are pending registrations awaiting review

- Pending registration count is highlighted.
- A direct link opens UC12.

Exceptions

E1 — Database error fetching stats

- Platform shows error state with retry option.

Technical Notes

- "Bookings this week" is a rolling 7-day window, not the calendar week — deliberately not reusing the Monday = 0 convention from availability_rules, since a trailing activity metric shouldn't reset to near-zero every Monday morning. A selectable custom date range is a deferred future improvement.
- GET /api/admin/me was added after the original admin routes were built, once the overview screen's identity block needed the logged-in admin's own name/email — none of the original routes returned it.

Use Case 14: Admin Views All Businesses

Actor	Platform Administrator (logged in via JWT)
Trigger	Admin clicks "Businesses" in the admin nav
Preconditions	Admin is authenticated with role = 'admin'.

Main Flow

- Admin clicks "Businesses".

- Platform fetches a list of every business in the system (no tenant filter — admin sees across all): business name, owner email, signup date, approval_status, is_active flag, and number of bookings.
- Platform renders a table with all businesses, paginated at 25 per page.
- Admin can search by business name or email, filter by approval_status and by is_active, and click a business to see details.

Postcondition: Admin has a complete view of every business on the platform across approval and activation status.

Alternate Flows

A1 — *Non-admin tries to access the admin businesses route directly*

- Backend checks JWT role — if not 'admin', returns 403 Forbidden.
- Redirects to login or business dashboard.

A2 — *Search returns no results*

- Platform shows an empty state explaining no businesses match the search.

Exceptions

E1 — *Database error during fetch*

- Platform shows error state with retry option.

Technical Notes

- This is the most academically meaningful endpoint of the admin role — it is the cross-tenant query that demonstrates Role-Based Access Control. Every other backend query in the system filters by business_id. This one deliberately doesn't — proving that the admin role grants legitimate cross-tenant access, in contrast to the 404-everywhere anti-enumeration pattern used for illegitimate cross-tenant attempts elsewhere in the system (see UC6 A1, UC9 A3).

Use Case 15: Admin Suspends or Activates a Business

Actor	Platform Administrator (logged in via JWT)
Trigger	Admin clicks the "Suspend" or "Activate" button next to a business in the businesses list (or from a business detail view)
Preconditions	Admin is authenticated with role = 'admin'. The target business exists and has approval_status = 'approved' (pending and rejected businesses are governed by UC12).

Main Flow

- Admin views the businesses list (UC14).
- Admin clicks "Suspend" (or "Activate" if the business is already suspended) on a specific business row.
- Platform shows a confirmation prompt appropriate to the action.
- Admin confirms the action.
- Platform updates the business's is_active flag (true → false for suspension, false → true for activation).
- Platform shows a success message confirming the new state.
- The business's row in the list reflects the new status.

Postcondition: Business's is_active flag is updated. The change immediately affects what the business can do.

Alternate Flows

A1 — *Admin cancels the confirmation prompt*

- Step 3: Admin cancels instead of confirming.
- No change is made.

A2 — Admin tries to suspend an already-suspended business

- The button shown should read "Activate", not "Suspend" — the UI prevents this state.
- If somehow attempted directly via API, the backend rejects it.

A3 — Non-admin tries to call the suspend/activate endpoint directly

- Backend checks JWT role — if not 'admin', returns 403 Forbidden.

Exceptions

E1 — Database error during status update

- Platform shows an error and asks the admin to try again.
- No partial update — atomic operation.

Technical Notes

- `is_active` enforcement happens at the middleware/guard layer: every protected owner route re-checks live `is_active` state on every request via `requireApprovedBusiness`, so a JWT that was valid at login can still be rejected mid-session if the business is suspended later.
- The public booking endpoint also checks `is_active` — it's a public route but still respects the suspended flag, otherwise customers could keep booking with a suspended business.
- Approved bookings are deliberately preserved when a business is suspended. Suspending a tenant should never invalidate user-facing commitments already made to customers.
- `approval_status` and `is_active` are two separate concerns. `approval_status` describes the lifetime gate (only set once per business by UC12). `is_active` is the ongoing operational toggle managed here.
- No email is sent to the business owner on suspension — silent by design, flagged as a possible future addition.
- No audit log is kept of suspension/reactivation actions. Flagged as a future improvement.

Effects of `approval_status` and `is_active` on Business Access

Area	Pending	Rejected	Suspended (<code>is_active=false</code>)	Active
Owner login	Blocked — "under review"	Blocked — rejection reason shown	Blocked — "account suspended"	Allowed
Public booking page	404 "Business not found."	404 "Business not found."	404 "Business not found."	Fully functional
Existing JWT	N/A (never had one)	N/A (never had one)	Next API request returns 401/403	Valid
Existing bookings	N/A	N/A	Preserved — not cancelled	Active
Widget on external site	Opens modal, shows not-available state	Opens modal, shows not-available state	Opens modal, shows not-available state	Fully functional

7. Revision Log

Corrections applied in this revision, reconciling the specification against the shipped codebase (docs/IMPLEMENTATION_NOTES.md, db_schema.sql, and the live routes):

UC	Was	Now
UC1, A2–A4	Three distinguishable messages for pending / rejected / suspended businesses	One identical 404 "Business not found." for all of A1–A4 (anti-enumeration)
UC2, A2	"Invalid value for [field label]"	"\${field.label} must be one of the provided options"
UC2 A3 / UC3 A1	Two different quoted messages for the same slot-race event	One string: "This slot was just taken. Please select another time." on both paths
UC5 (whole use case)	Dedicated password_reset_tokens table, hashed tokens, used flag, 1-hour expiry, multi-token invalidation logic	Two plain columns on users (reset_token, reset_token_expires_at), 30-minute expiry, single-token-by-construction
UC6, A1	Cross-tenant booking action → 403 Forbidden	404 Not Found, everywhere, deliberately (anti-enumeration)
UC9, A3	Cross-tenant booking access → 403 Forbidden	404 Not Found — the exact passage IMPLEMENTATION_NOTES.md flags by name
UC10, Section B + Technical Notes	iframe HTML snippet; notes on iframe height/scroll/postMessage resize	<script> tag opening a Shadow-DOM modal widget (widget.js); iframe notes removed as no longer applicable
UC10, A1	Live public page itself shows the "no availability" message	Share & Embed screen checks GET /owner/availability on click and shows a dismissible banner before opening the live page

Revision note: UC numbering itself was not changed — this document's numbering (UC1–UC15 as indexed in Section 3) is what IMPLEMENTATION_NOTES.md already references and is treated as authoritative. The companion API Endpoint Reference document has been updated separately to point at these same UC numbers, since it had drifted onto an older numbering scheme.

End of document

Hussain Jameel · ITA602 · Torrens University Australia